



Mestrado em Engenharia Informática
Cibersegurança e Administração de Sistemas

MarkDawn Phase 2 - Sprint 1

Relatório

Autores:

Abílio Correia 1212111
Eduardo Sampaio 1250511
Eduardo Vasconcelos 1250510
Diogo Melo 1250506

Desenvolvimento de Software Seguro

2025/2026

Índice

1	Introdução	6
2	Arquitetura Docker do Projeto	7
2.1	Organização dos Serviços	7
2.2	Serviço Web (Flask API)	7
2.3	Serviço Workspace	8
2.4	Serviço de Base de Dados	8
2.5	Serviço de Backup	8
2.6	Persistência de Dados	9
3	Software Composition Analysis	10
3.1	Objetivo	10
3.2	Ferramentas utilizadas	10
3.3	Análise da árvore de dependências	10
3.3.1	Resultados obtidos	11
3.3.2	Classificação de risco das dependências	12
3.4	Geração de Software Bill of Materials	13
3.4.1	Objetivo do SBOM	13
3.4.2	Resultados principais	13
3.5	Análise de pacotes desatualizados	15
3.5.1	Resultados obtidos	15
3.6	Análise de vulnerabilidades com pip-audit	15
3.6.1	Execução da ferramenta	16
3.6.2	Resultados obtidos	16

3.7	Validação adicional de vulnerabilidades com Safety	16
3.7.1	Execução da ferramenta	17
3.7.2	Resultados obtidos	17
3.8	Riscos associados à cadeia de dependências	18
3.9	Integração futura em CI/CD	18
3.10	Conclusão SCA	18
4	Software Composition Analysis - Docker	19
4.1	Objetivo	19
4.2	Ferramentas utilizadas	19
4.3	Análise da imagem Docker Web	19
4.3.1	Resultados obtidos	19
4.3.2	Interpretação dos resultados	20
4.3.3	Mitigação e boas práticas	20
4.4	Análise da imagem MySQL	21
4.4.1	Resultados obtidos	21
4.5	Análise de Dockerfiles	22
4.5.1	Resultados obtidos	22
4.5.2	Erro de configuração - Backup	22
4.5.3	Erro de configuração - Web	23
4.5.4	Erro de configuração - Workspace	23
4.5.5	Interpretação dos resultados	23
4.6	Conclusão da análise Docker SCA	24
5	Implementação de SAST e DAST	25

5.1	Static Application Security Testing (SAST)	25
5.2	Dynamic Application Security Testing (DAST)	26
5.3	Conclusão	28
6	Conformidade com ASVS 5.0	29
6.1	Validação de Inputs e lógica de negócio	29
6.2	Gestão de Sessões	31
7	Conclusão	33

Índice de imagens

1	Excerto representativo da árvore de dependências principal do projeto	11
2	Excerto representativo do Software Bill of Materials (SBOM)	14
3	Resultados da análise de pacotes desatualizados	15
4	Execução da ferramenta pip-audit	16
5	Resultado da análise de vulnerabilidades com pip-audit	16
6	Execução da ferramenta Safety	17
7	Resultado da análise de vulnerabilidades com Safety	17
8	Excerto representativo do workflow SAST	25
9	Excerto representativo dos resultados Semgrep	26
10	Excerto representativo do workflow DAST	27
11	Excerto representativo dos resultados OWASP ZAP	28

Índice de tabelas

1	Classificação de risco das dependências do projeto	12
2	Análise detalhada de vulnerabilidades na imagem Docker utilizando Trivy	19
3	Resultados da análise de vulnerabilidades da imagem Docker MySQL utilizando Trivy	21
4	Resumo das misconfigurações detetadas nos ficheiros Docker	22
5	Tabela de Validação de Dados	29
6	Restrições operacionais por utilizador	30
7	Restrições globais da aplicação	31
8	Tabela de Controlo de Sessão e Expiração de Tokens	31
9	Tabela de Controlo de Sessões Ativas Simultâneas	32

1 Introdução

O presente documento apresenta a análise de segurança realizada ao projeto, abrangendo diferentes camadas do ciclo de desenvolvimento de software. A avaliação inclui análise estática de código (SAST), análise dinâmica (DAST), Software Composition Analysis (SCA), configuração de containers Docker e verificação de conformidade com o padrão ASVS.

O objetivo é identificar vulnerabilidades, más práticas de implementação e configurações inseguras, bem como avaliar o nível de maturidade de segurança da aplicação de forma global.

2 Arquitetura Docker do Projeto

Para permitir a execução local da aplicação de forma simples, consistente e isolada, foi desenvolvida uma arquitetura baseada em Docker.

A solução implementada permite separar os diferentes componentes do sistema em serviços independentes, cada um executado no seu próprio container.

A configuração principal da infraestrutura encontra-se no ficheiro:

```
docker/docker-compose.yml
```

2.1 Organização dos Serviços

A arquitetura encontra-se dividida em quatro serviços principais:

- API principal desenvolvida em Flask;
- Serviço Workspace;
- Base de dados MySQL;
- Serviço dedicado a backups.

2.2 Serviço Web (Flask API)

O serviço **web** corresponde à API principal da aplicação, responsável pela lógica de negócio e comunicação com a base de dados e workspaces.

A configuração deste serviço encontra-se no ficheiro:

```
docker/web/Dockerfile
```

Este container executa a aplicação Flask e disponibiliza a API localmente através de uma porta dedicada.

Foram também utilizadas variáveis de ambiente para configuração dos parâmetros da base de dados e comunicação entre serviços, permitindo separar configurações sensíveis do código-fonte.

2.3 Serviço Workspace

O serviço **workspace** foi desenvolvido como um componente independente responsável pela gestão dos workspaces dos utilizadores.

A configuração do container encontra-se em:

```
docker/workspace/Dockerfile
```

A separação deste serviço da API principal permite modularizar funcionalidades específicas e reduzir o acoplamento entre componentes.

Foi ainda configurado armazenamento persistente para manter os dados dos workspaces mesmo após reinícios dos containers.

2.4 Serviço de Base de Dados

A base de dados é executada através de um container MySQL independente.

Foi utilizada persistência através de volumes Docker, garantindo que os dados permanecem armazenados independentemente do ciclo de vida dos containers.

Além disso, foi configurado um script de inicialização automática da base de dados, localizado em:

```
docker/mysql/init.sql
```

Este script permite automatizar a criação inicial da estrutura da base de dados.

2.5 Serviço de Backup

Foi igualmente desenvolvido um serviço dedicado à realização de backups automáticos da aplicação.

A configuração deste serviço encontra-se em:

```
docker/backup/Dockerfile
```

Este container possui acesso aos volumes persistentes da base de dados e dos workspaces, permitindo centralizar operações de backup e recuperação de dados.

2.6 Persistência de Dados

Para garantir persistência de informação, foram utilizados volumes Docker para:

- Dados da base de dados;
- Ficheiros dos workspaces;
- Armazenamento dos backups.

Esta abordagem permite manter os dados mesmo após reinícios ou recriação dos containers.

3 Software Composition Analysis

3.1 Objetivo

O Software Composition Analysis foi realizado com o objetivo de identificar, analisar e avaliar riscos de segurança presentes nas dependências do projeto, incluindo vulnerabilidades conhecidas, dependências desatualizadas e a estrutura completa de dependências da aplicação.

Esta análise é essencial em aplicações modernas, uma vez que grande parte das vulnerabilidades provém de bibliotecas externas e não diretamente do código desenvolvido.

3.2 Ferramentas utilizadas

Foram utilizadas várias ferramentas para fazer uma análise abrangente:

- **pipdeptree** – Análise da árvore de dependências
- **pip list --outdated** – Verificação de versões desatualizadas
- **pip-audit** – Detecção de vulnerabilidades conhecidas (CVEs)
- **safety** – Validação independente de vulnerabilidades em dependências

Estas ferramentas foram executadas em ambiente isolado, garantindo que a análise reflete apenas as dependências do projeto.

3.3 Análise da árvore de dependências

A ferramenta **pipdeptree** foi utilizada para visualizar a estrutura completa de dependências do projeto, incluindo dependências diretas e transitivas.

Esta análise permitiu:

- Identificar todas as bibliotecas utilizadas indiretamente
- Compreender a complexidade da cadeia de dependências
- Mapear potenciais superfícies de ataque indiretas

3.3.1 Resultados obtidos

A Figura 1 apresenta um excerto representativo da árvore de dependências gerada pela ferramenta pipdeptree, incluindo dependências diretas e transitivas relevantes para a aplicação.

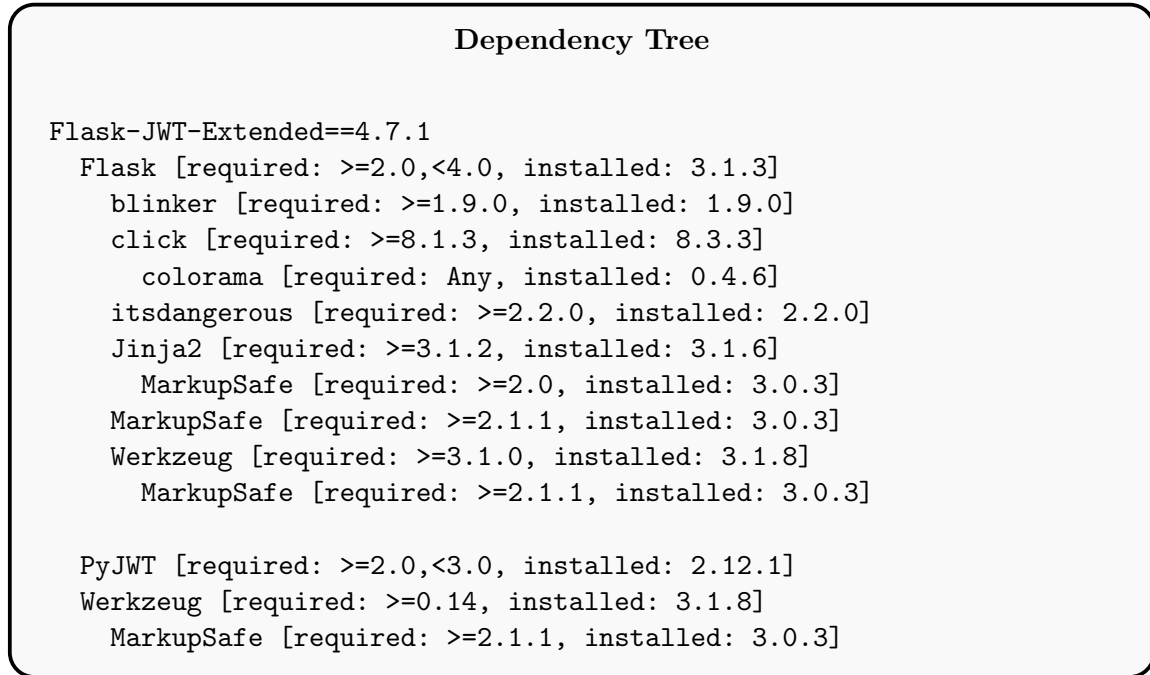


Figura 1: Excerto representativo da árvore de dependências principal do projeto

O excerto acima apresentado representa os principais componentes utilizados pela aplicação, incluindo o framework web Flask, autenticação JWT, acesso à base de dados e bibliotecas auxiliares utilizadas durante o desenvolvimento e testes.

A análise revelou uma estrutura típica de aplicações Flask, com dependências como Werkzeug, Jinja2 e SQLAlchemy presentes como dependências transitivas.

3.3.2 Classificação de risco das dependências

Foi identificado que as dependências críticas da aplicação incluem o framework Flask, a biblioteca de autenticação JWT e os drivers de acesso à base de dados. Estas bibliotecas são consideradas de maior impacto em caso de vulnerabilidade, uma vez que estão diretamente ligadas à lógica central da aplicação.

Categoria	Biblioteca	Impacto
Framework	Flask	Alto
Autenticação	PyJWT	Alto
Base Dados	SQLAlchemy	Alto
Utils	click, Jinja2	Médio

Tabela 1: Classificação de risco das dependências do projeto

A classificação de risco das dependências foi realizada com base no impacto potencial de comprometimento de cada biblioteca no contexto da aplicação, considerando critérios de segurança e arquitetura de software, nomeadamente:

- **Superfície de ataque** – Exposição a input não confiável, nomeadamente através de requisições HTTP, parsing de dados ou interação com utilizador;
- **Acesso a dados sensíveis** – Capacidade da biblioteca de interagir com informação crítica, como autenticação, tokens ou base de dados;
- **Impacto de exploração** – Consequências de uma vulnerabilidade, incluindo possibilidade de execução remota de código, escalonamento de privilégios ou comprometimento de dados;
- **Dependência no fluxo de execução** – Grau em que a aplicação depende diretamente da biblioteca para funcionalidades essenciais;
- **Natureza da dependência** – Distinção entre dependências diretas (core da aplicação) e transitivas (dependências indiretas de suporte).
- **Centralidade na arquitetura** – Importância da biblioteca no fluxo principal da aplicação;

Com base nestes critérios, foi atribuída uma classificação qualitativa de impacto a cada categoria de dependências.

3.4 Geração de Software Bill of Materials

De forma a complementar a análise da estrutura de dependências, foi gerado um SBOM utilizando o standard CycloneDX.

Este artefacto fornece uma representação estruturada de todas as dependências da aplicação, incluindo versões, identificadores únicos (PURL) e relações entre componentes.

O SBOM permite formalizar a lista de dependências identificadas na análise anterior, servindo como um inventário completo da cadeia de supply chain do software.

3.4.1 Objetivo do SBOM

O principal objetivo da geração do SBOM é garantir transparência e rastreabilidade das dependências da aplicação, permitindo:

- Inventariar todas as bibliotecas utilizadas;
- Facilitar auditorias de segurança;
- Suportar integração futura com ferramentas de CI/CD;
- Melhorar a detecção de vulnerabilidades na cadeia de dependências.

3.4.2 Resultados principais

O SBOM gerado contém 22 componentes, correspondendo a todas as dependências definidas no ficheiro **requirements.txt**.

Não foram identificadas dependências ocultas ou não declaradas, o que indica controlo total sobre a cadeia de dependências da aplicação.

Todos os componentes incluem versão fixa e identificador PURL, garantindo rastreabilidade e reprodutibilidade do ambiente.

A Figura 2 apresenta um excerto representativo do ficheiro SBOM gerado em formato CycloneDX.

```
Software Bill of Materials (CycloneDX)

{
  "components": [
    {
      "bom-ref": "requirements-L4",
      "description": "requirements line 4: Flask==3.1.3",
      "externalReferences": [
        {
          "comment": "implicit dist url",
          "type": "distribution",
          "url": "requirements.txt/Flask/"
        }
      ],
      "name": "Flask",
      "purl": "pkg:pypi/flask@3.1.3",
      "type": "library",
      "version": "3.1.3"
    }
  ]
}
```

Figura 2: Excerto representativo do Software Bill of Materials (SBOM)

3.5 Análise de pacotes desatualizados

Foi realizada uma análise de versões desatualizadas através do comando **pip list --outdated**, com o objetivo de identificar bibliotecas que possuem versões mais recentes disponíveis.

Esta análise permite avaliar a maturidade de manutenção das dependências e identificar potenciais riscos associados ao uso de versões antigas, mesmo quando não existem vulnerabilidades conhecidas associadas.

3.5.1 Resultados obtidos

A execução do comando permitiu identificar os seguintes resultados:

Pacotes desatualizados detetados		
Package	Version	Latest
-----	-----	-----
click	8.3.3	8.4.0
Flask-JWT-Extended	4.7.1	4.7.4

Figura 3: Resultados da análise de pacotes desatualizados

Os resultados indicam que existem apenas duas dependências com versões mais recentes disponíveis. No entanto, não foram identificadas vulnerabilidades associadas às versões atualmente utilizadas.

Apesar de não existir impacto de segurança imediato, a atualização regular de dependências é recomendada como boa prática de segurança, uma vez que novas versões podem incluir correções de bugs, melhorias de performance e correções de vulnerabilidades futuras.

3.6 Análise de vulnerabilidades com pip-audit

Foi utilizada a ferramenta **pip-audit** para realizar uma análise automática de vulnerabilidades conhecidas nas dependências do projeto.

A ferramenta consulta bases de dados públicas de vulnerabilidades, permitindo identificar bibliotecas afetadas por falhas de segurança conhecidas.

Esta análise é particularmente importante em ambientes DevSecOps, uma vez que aplicações modernas dependem fortemente de bibliotecas externas, podendo herdar vulnerabilidades presentes nessas dependências.

3.6.1 Execução da ferramenta

A análise foi realizada através do seguinte comando:

```
pip-audit -r requirements.txt
```

Figura 4: Execução da ferramenta pip-audit

3.6.2 Resultados obtidos

O resultado da análise foi o seguinte:

Resultado da análise pip-audit

```
No known vulnerabilities found
```

Figura 5: Resultado da análise de vulnerabilidades com pip-audit

A ferramenta não identificou vulnerabilidades conhecidas nas dependências atualmente utilizadas pelo projeto.

Este resultado indica que as bibliotecas analisadas não possuem CVEs publicamente conhecidas no momento da execução da análise.

A utilização do **pip-audit** permitiu validar a integridade do conjunto de dependências da aplicação, reduzindo a exposição a vulnerabilidades introduzidas por dependências externas.

3.7 Validação adicional de vulnerabilidades com Safety

Para complementar a análise realizada com **pip-audit**, foi utilizada a ferramenta **Safety** como mecanismo adicional de validação de vulnerabilidades nas dependências do projeto.

A utilização de múltiplas ferramentas de análise permite aumentar a confiança nos resultados obtidos, uma vez que diferentes ferramentas podem recorrer a bases de dados e metodologias distintas para detecção de vulnerabilidades.

3.7.1 Execução da ferramenta

A análise foi executada através do seguinte comando:

```
safety check -r requirements.txt
```

Figura 6: Execução da ferramenta Safety

3.7.2 Resultados obtidos

O resultado da análise foi o seguinte:

```
Resultado da análise Safety

Safety v3.7.0 - Vulnerability Scan

Scanning requirements.txt...

Found and scanned 22 packages

Timestamp: 2026-05-17 14:41:38

0 vulnerabilities reported
0 vulnerabilities ignored

No known security vulnerabilities reported.
```

Figura 7: Resultado da análise de vulnerabilidades com Safety

A ferramenta **Safety** confirmou a ausência de vulnerabilidades conhecidas nas dependências analisadas.

Esta validação adicional reforça a confiança nos resultados previamente obtidos com **pip-audit**, demonstrando consistência entre diferentes mecanismos de análise de segurança.

Além da detecção de vulnerabilidades, esta abordagem contribui para melhorar a rastreabilidade e monitorização contínua da segurança da cadeia de dependências da aplicação.

3.8 Riscos associados à cadeia de dependências

Apesar de não terem sido identificadas vulnerabilidades conhecidas nas dependências analisadas, a cadeia de supply chain de software continua a representar um vetor de risco relevante.

Dependências de terceiros podem introduzir vulnerabilidades ao longo do tempo, sofrer alterações incompatíveis entre versões ou, em cenários mais críticos, ser alvo de comprometimento malicioso.

Desta forma, torna-se essencial a monitorização contínua da cadeia de dependências ao longo do ciclo de vida da aplicação, especialmente em projetos com atualização frequente de bibliotecas.

3.9 Integração futura em CI/CD

Como evolução futura, as ferramentas utilizadas nesta análise poderão ser integradas na pipeline de CI/CD, permitindo a execução automática de verificações de segurança sempre que existam alterações às dependências do projeto.

Esta abordagem permite a deteção precoce de vulnerabilidades introduzidas por novas versões de bibliotecas, promovendo um modelo de segurança contínua alinhado com práticas de DevSecOps.

Adicionalmente, a automatização destas verificações contribui para reduzir a probabilidade de introdução de dependências inseguras em produção.

3.10 Conclusão SCA

A análise de Software Composition Analysis permitiu concluir que, no momento da avaliação, o projeto não apresenta vulnerabilidades conhecidas nas suas dependências externas.

As ferramentas utilizadas permitiram obter uma visão abrangente da cadeia de dependências, incluindo estrutura transitiva, estado de atualização e validação de vulnerabilidades conhecidas através de múltiplos motores de análise.

Apesar da existência de algumas bibliotecas com versões mais recentes disponíveis, não foram identificados riscos críticos associados às versões atualmente utilizadas.

De forma global, os resultados indicam uma cadeia de dependências estável e controlada, alinhada com boas práticas de DevSecOps e com uma exposição reduzida a vulnerabilidades conhecidas.

4 Software Composition Analysis - Docker

4.1 Objetivo

De forma a complementar a análise de Software Composition Analysis realizada ao nível das dependências da aplicação, foi efetuada uma análise adicional às imagens Docker utilizadas no projeto.

Esta análise tem como objetivo identificar vulnerabilidades presentes no sistema operativo base das imagens, bem como nas dependências instaladas dentro dos containers, permitindo avaliar o risco associado à cadeia de supply chain ao nível da infraestrutura.

4.2 Ferramentas utilizadas

Para esta análise foi utilizada a ferramenta **Trivy**, uma solução amplamente utilizada para deteção de vulnerabilidades em imagens Docker, ficheiros de dependências e repositórios.

A ferramenta permite identificar vulnerabilidades conhecidas tanto ao nível do sistema operativo como das bibliotecas instaladas.

4.3 Análise da imagem Docker Web

A imagem principal da aplicação Flask foi analisada utilizando o Trivy, tendo sido identificado o sistema base **Debian 13.4** e respetivas dependências Python instaladas no ambiente do container.

A análise permitiu identificar vulnerabilidades em diferentes componentes da imagem, incluindo o sistema operativo e pacotes instalados via pip.

4.3.1 Resultados obtidos

A Tabela 2 apresenta um resumo dos resultados obtidos através da execução do Trivy sobre a imagem Docker da aplicação.

Target	Type	Total	Unknown	Low	Medium	High	Critical
docker-web (debian 13.4)	debian	1673	33	985	484	150	21
app/venv Python packages	python-pkg	0	0	0	0	0	0
usr/local Python packages	python-pkg	7	0	0	0	5	2

Tabela 2: Análise detalhada de vulnerabilidades na imagem Docker utilizando Trivy

4.3.2 Interpretação dos resultados

Os resultados obtidos demonstram que a maior parte das vulnerabilidades não está diretamente relacionada com o código da aplicação, mas sim com a imagem base utilizada.

Este é um comportamento esperado em ambientes Docker, especialmente quando são utilizadas imagens genéricas de sistemas operativos como Debian.

As vulnerabilidades identificadas em pacotes Python são reduzidas, o que indica um controlo relativamente eficaz das dependências da aplicação.

4.3.3 Mitigação e boas práticas

Com base nos resultados obtidos, podem ser consideradas as seguintes boas práticas:

- Utilização de imagens base mais minimalistas;
- Atualização regular das imagens base utilizadas;
- Redução do número de pacotes instalados no container;
- Utilização de versões fixas e seguras das dependências;
- Integração contínua do Trivy na pipeline CI/CD.

4.4 Análise da imagem MySQL

De forma a complementar a análise de Software Composition Analysis ao nível das imagens Docker, foi realizada uma avaliação à imagem **mysql:8**, utilizada no serviço de base de dados da aplicação.

Esta análise teve como objetivo identificar vulnerabilidades presentes na imagem oficial.

4.4.1 Resultados obtidos

A Tabela 3 apresenta o resumo da análise realizada à imagem MySQL.

Target	Type	Vulnerabilities	Secrets
mysql:8 (Oracle 9.7)	OS / Container	0	-
Python packages (mysqlsh)	python-pkg	10	-
gosu binary	gobinary	33	-

Tabela 3: Resultados da análise de vulnerabilidades da imagem Docker MySQL utilizando Trivy

A análise da imagem MySQL revelou um nível de vulnerabilidades significativamente mais reduzido quando comparado com a imagem da aplicação web.

Foram detetadas vulnerabilidades em componentes adicionais incluídos na imagem, nomeadamente bibliotecas Python associadas ao **MySQL Shell** e no binário **gosu**.

Entre as vulnerabilidades identificadas destacam-se problemas em bibliotecas como **cryptography**, **paramiko** e **urllib3**, embora nenhuma destas seja classificada como crítica.

No total, foram identificadas vulnerabilidades distribuídas maioritariamente pelos níveis low, medium e high, sem existência de vulnerabilidades críticas.

Este resultado demonstra que, apesar de se tratar de uma imagem oficial, continuam a existir componentes com vulnerabilidades conhecidas, o que reforça a importância da análise contínua de imagens base utilizadas em ambientes containerizados.

4.5 Análise de Dockerfiles

De forma a complementar a análise de Software Composition Analysis ao nível da infraestrutura containerizada, foi realizada uma análise adicional a todos os ficheiros Docker presentes no diretório `/docker`, incluindo **Dockerfiles** e o ficheiro **docker-compose**.

Esta análise teve como objetivo identificar configurações incorretas de segurança e boas práticas não seguidas na definição dos containers, permitindo avaliar riscos ao nível da configuração da infraestrutura.

4.5.1 Resultados obtidos

A Tabela 4 apresenta o resumo dos resultados obtidos na análise a todos os ficheiros Docker do projeto.

Target	Type	Misconfigurations
backup/Dockerfile	dockerfile	3
web/Dockerfile	dockerfile	3
workspace/Dockerfile	dockerfile	3

Tabela 4: Resumo das misconfigurações detetadas nos ficheiros Docker

No total foram identificadas **9 erros de configuração**, distribuídos uniformemente pelos diferentes Dockerfiles analisados.

Todas as análises apresentaram resultados consistentes, indicando padrões repetidos de configuração entre os diferentes serviços da aplicação.

4.5.2 Erro de configuração - Backup

O ficheiro **backup/Dockerfile** apresentou três misconfigurações de segurança, abrangendo diferentes níveis de severidade.

Os seguintes erros de configuração foram identificados:

- **DS-0002 (High)** – Execução do container como utilizador root, sem definição de utilizador não privilegiado;
- **DS-0026 (Low)** – Ausência de instrução HEALTHCHECK;
- **DS-0029 (High)** – Falta da flag `--no-install-recommends` na instalação de pacotes via apt-get.

4.5.3 Erro de configuração - Web

O ficheiro **web/Dockerfile** apresentou um conjunto semelhante de misconfigurações ao ficheiro anterior, evidenciando reutilização de padrões de configuração.

Os seguintes erros de configuração foram identificados:

- **DS-0002 (High)** – Container executado como utilizador root;
- **DS-0026 (Low)** – Ausência de instrução HEALTHCHECK;
- **DS-0029 (High)** – Falta da flag `--no-install-recommends` na instalação de dependências.

4.5.4 Erro de configuração - Workspace

O ficheiro **workspace/Dockerfile** apresenta o mesmo padrão de misconfigurações identificado nos restantes ficheiros analisados.

Os seguintes erros de configuração foram identificados:

- **DS-0002 (High)** – Execução como utilizador root;
- **DS-0026 (Low)** – Ausência de HEALTHCHECK;
- **DS-0029 (High)** – Falta da flag `--no-install-recommends`.

4.5.5 Interpretação dos resultados

Os resultados obtidos demonstram a existência de padrões consistentes de erros de configuração ao longo dos diferentes Dockerfiles do projeto.

As principais falhas identificadas estão relacionadas com:

- Execução de containers com utilizador **root**;
- Ausência de mecanismos de monitorização do estado das máquinas;
- Instalação de dependências sem otimizações de segurança e tamanho de imagem.

Estas configurações não introduzem vulnerabilidades diretas ao nível de CVEs, mas aumentam a superfície de ataque da infraestrutura e reduzem o nível de hardening das imagens.

4.6 Conclusão da análise Docker SCA

A análise de erros de configuração em ficheiros Docker permitiu identificar um conjunto consistente de boas práticas não aplicadas na definição das imagens do projeto.

Embora não tenham sido detetadas vulnerabilidades críticas diretamente exploráveis ao nível de CVEs, foram identificadas várias configurações inseguras que podem impactar a segurança da aplicação em execução.

De forma global, os resultados evidenciam a necessidade de reforçar o hardening das imagens Docker, especialmente através da utilização de utilizadores não privilegiados, melhoria da gestão do ciclo de vida dos containers e otimização das instruções de build.

A integração destas verificações em pipelines de CI/CD é recomendada, de forma a garantir que futuras alterações aos Dockerfiles não introduzem novas misconfigurações de segurança.

5 Implementação de SAST e DAST

No contexto do desenvolvimento seguro da aplicação, foram integrados mecanismos automáticos de análise de segurança no pipeline de CI/CD através do GitHub Actions. O principal objetivo desta integração consistiu em introduzir validações contínuas de segurança ao longo do ciclo de desenvolvimento, permitindo identificar vulnerabilidades, más práticas de programação e configurações inseguras ainda durante as fases iniciais do projeto.

Para este efeito foram utilizadas duas abordagens complementares: *Static Application Security Testing (SAST)* e *Dynamic Application Security Testing (DAST)*. Enquanto o SAST permite analisar o código-fonte e ficheiros de configuração sem necessidade de executar a aplicação, o DAST realiza testes dinâmicos sobre a aplicação em execução, simulando interações reais sobre os seus endpoints HTTP.

5.1 Static Application Security Testing (SAST)

A implementação de SAST teve como objetivo realizar análise estática ao código-fonte da aplicação e aos ficheiros de configuração associados ao projeto. Esta abordagem permitiu identificar potenciais vulnerabilidades e más práticas de desenvolvimento antes da execução da aplicação, reduzindo o risco de propagação de problemas de segurança para fases posteriores do desenvolvimento.

Para a implementação desta componente foram utilizadas as ferramentas *Bandit* e *Semgrep*. O Bandit foi utilizado para análise específica de aplicações Python, permitindo identificar padrões inseguros relacionados com o backend Flask desenvolvido no projeto. Já o Semgrep foi configurado com regras associadas à OWASP Top 10, permitindo análise multicamada sobre ficheiros Python, Dockerfiles, YAML e outros componentes da infraestrutura.

A execução do SAST foi integrada automaticamente no GitHub Actions através de um workflow dedicado. Sempre que ocorre um *push* ou *pull request*, a pipeline realiza automaticamente o checkout do repositório, configura o ambiente Python, instala as dependências necessárias e executa as ferramentas de análise estática.

A Figura 8 apresenta um excerto representativo da configuração utilizada no workflow SAST.

```
- name: Run Bandit
  run: |
    bandit -r . -f txt -o bandit-report.txt || true

- name: Run Semgrep
  run: |
    semgrep --config=p/owasp-top-ten . || true
```

Figura 8: Excerto representativo do workflow SAST

Durante a execução do Semgrep foram analisados 36 ficheiros do projeto utilizando 241 regras de segurança. A análise identificou cinco findings classificados como *blocking*. :contentReference[oaicite:0]index=0

Entre os principais problemas identificados destacam-se a execução da aplicação Flask com `debug=True`, a exposição da aplicação através do host `0.0.0.0` e a execução dos containers Docker utilizando o utilizador `root`. Foram igualmente identificadas configurações relacionadas com a ausência de utilizadores não privilegiados nos Dockerfiles da aplicação.

A Figura 9 apresenta um excerto representativo dos resultados obtidos durante a execução do Semgrep.

```
app.py

Detected Flask app with debug=True.
Running flask app with host 0.0.0.0
could expose the server publicly.

docker/web/Dockerfile

By not specifying a USER, a program in
the container may run as 'root'.
```

Figura 9: Excerto representativo dos resultados Semgrep

Relativamente ao Bandit, não foram identificadas vulnerabilidades críticas adicionais durante a análise realizada. :contentReference[oaicite:1]index=1

Os resultados obtidos demonstraram a utilidade da análise estática na identificação precoce de más práticas de configuração e potenciais riscos de segurança. A utilização de `debug=True` em aplicações Flask representa um risco relevante, uma vez que pode expor informação interna da aplicação e mensagens detalhadas de erro. Da mesma forma, a execução de containers Docker como utilizador `root` constitui uma prática insegura, aumentando o impacto potencial de um eventual comprometimento do container.

5.2 Dynamic Application Security Testing (DAST)

Para complementar a análise estática foi igualmente implementado um mecanismo de *Dynamic Application Security Testing (DAST)* utilizando a ferramenta *OWASP ZAP (Zed Attack Proxy)*.

Ao contrário do SAST, esta abordagem realiza testes dinâmicos sobre a aplicação em execução, permitindo identificar problemas relacionados com configuração HTTP, ausência de mecanismos de hardening e potenciais exposições de informação sensível.

A implementação do DAST foi realizada através de um workflow GitHub Actions dedicado, utili-

zando Docker Compose para criar automaticamente um ambiente de execução contendo a aplicação Flask e a base de dados MySQL.

Durante a execução do workflow, é criado dinamicamente um ficheiro `.env` utilizando GitHub Secrets, permitindo configurar de forma segura as variáveis de ambiente necessárias à execução da aplicação. Após o arranque dos containers Docker, o workflow aguarda até que a API Flask fique operacional antes de iniciar o scan automático com OWASP ZAP.

A Figura 10 apresenta um excerto representativo da configuração utilizada no workflow DAST.

```
- name: OWASP ZAP Baseline Scan
  continue-on-error: true
  uses: zaproxy/action-baseline@v0.12.0
  with:
    target: 'http://localhost:5003'
    allow_issue_writing: false
    cmd_options: '-I'
```

Figura 10: Excerto representativo do workflow DAST

Durante a implementação desta componente surgiram problemas relacionados com sincronização entre os serviços Docker, particularmente entre a aplicação Flask e a base de dados MySQL. O serviço Flask tentava estabelecer ligação à base de dados antes da conclusão do processo de inicialização do MySQL, originando falhas durante a execução da pipeline CI/CD.

Para resolver este problema foi implementado um mecanismo de *retry* na ligação à base de dados, permitindo que a aplicação aguardasse até o serviço MySQL ficar operacional antes de continuar a execução.

Após estabilização da infraestrutura Docker, o OWASP ZAP executou corretamente a análise dinâmica sobre os endpoints HTTP da aplicação. `:contentReference[oaicite:2]index=2`

Os resultados obtidos demonstraram a ausência de vulnerabilidades críticas, tendo sido identificados 12 warnings e 55 verificações concluídas com sucesso.

Entre os principais problemas identificados destacam-se a ausência de vários headers de segurança HTTP, nomeadamente `X-Frame-Options`, `Content-Security-Policy` (CSP), `X-Content-Type-Options` e `Permissions-Policy`. Foram igualmente identificadas situações relacionadas com possível exposição de informação sensível em URLs e divulgação de informação associada ao servidor HTTP utilizado.

A Figura 11 apresenta um excerto representativo dos resultados obtidos através do OWASP ZAP.

```
WARN-NEW: Missing Anti-clickjacking Header
WARN-NEW: X-Content-Type-Options Header Missing
WARN-NEW: Content Security Policy (CSP) Header Not Set
WARN-NEW: Permissions Policy Header Not Set
WARN-NEW: Information Disclosure - Sensitive Information in URL
```

Figura 11: Excerto representativo dos resultados OWASP ZAP

Os resultados obtidos demonstraram a importância da análise dinâmica na identificação de problemas relacionados com hardening da aplicação e configuração HTTP. A ausência destes headers de segurança pode facilitar ataques como clickjacking, content injection ou MIME sniffing, comprometendo a robustez da aplicação em ambientes reais.

5.3 Conclusão

A integração de mecanismos automáticos de SAST e DAST permitiu introduzir práticas contínuas de validação de segurança ao longo do desenvolvimento da aplicação.

A utilização de análise estática possibilitou identificar configurações inseguras e más práticas de desenvolvimento ainda antes da execução da aplicação, reduzindo o risco de propagação de vulnerabilidades para ambientes de teste ou produção.

Por outro lado, a análise dinâmica realizada com OWASP ZAP permitiu avaliar o comportamento da aplicação em runtime, identificando problemas relacionados com hardening HTTP, exposição de informação e ausência de mecanismos adicionais de proteção ao nível dos headers de segurança.

A automatização destas verificações através de GitHub Actions contribuiu para a criação de uma pipeline CI/CD mais alinhada com práticas modernas de DevSecOps, permitindo detecção precoce de vulnerabilidades, melhoria contínua da segurança e maior rastreabilidade dos resultados obtidos.

Apesar de não terem sido identificadas vulnerabilidades críticas, os resultados demonstraram a existência de diversas melhorias de segurança que deverão ser implementadas futuramente, nomeadamente ao nível da configuração da aplicação Flask, isolamento de containers Docker e reforço dos mecanismos de hardening HTTP.

De forma global, a implementação de SAST e DAST contribuiu significativamente para aumentar a maturidade de segurança do processo de desenvolvimento do projeto.

6 Conformidade com ASVS 5.0

Este capítulo apresenta a descrição dos métodos e controlos de conformidade implementados no sistema em relação aos requisitos estabelecidos pelo Application Security Verification Standard (ASVS 5.0) da OWASP.

6.1 Validação de Inputs e lógica de negócio

Esta secção incide sobre a validação de inputs para mitigar vulnerabilidades estruturais antes da persistência ou processamento de dados. Para o efeito, adota uma abordagem defensiva fundamentada no princípio do Allow-List e na validação rigorosa ao nível do servidor. Nas secções seguintes, detalham-se os requisitos formais de validação, abrangendo desde os mecanismos de autenticação e proteção de credenciais até às estratégias futuras de controlo do sistema de ficheiros.

v2.1.1

Tabela 5: Tabela de Validação de Dados

Input	Tipo de Dados	Compr Mínimo	Compr Máximo	Formato / Char Permitidos	Forma de Validação
Username	String	6 caracteres	50 caracteres	Letras, números e underscores	Remoção de espaços (<code>strip</code>) + Regex Match.
Password	String (Plaintext)	8 caracteres	Sem limite teórico	Todos os caracteres UTF-8 são aceites	Verificação de comprimento + Controlo contra o dicionário <code>rockyou.txt</code> .
Caminho do Ficheiro (Path)	String / Path	1 caractere	255 caracteres	Alfanuméricos, hífen, underscore e barra de diretório	Rejeição estrita de <code>.</code> e <code>...</code> . Resolução de caminho absoluto via <code>pathlib</code> comparado contra uma diretoria base restrita (Chroot lógico).
Conteúdo Markdown	String / Texto	0 caracteres	Limite definido por quota (ex: 2MB)	Caracteres UTF-8 (Texto Markdown válido)	Processamento e sanitização obrigatória na renderização de HTML para neutralizar tags executáveis (XSS).

v2.1.2

Em conformidade com o requisito v2.1.2 do ASVS 5.0, o sistema não se limita a validar dados de forma isolada. No contexto desta aplicação, esta validação foca-se na seguinte trindade de segurança: Identidade do Utilizador, Identificador do Workspace e Caminho do Ficheiro. O sistema deve garantir, programaticamente e antes de qualquer operação de I/O, que o documento solicitado pertence logicamente ao ecossistema (Workspace) mapeado para o utilizador autenticado, impedindo ataques de path travessal e IDOR (Insecure Direct Object References).

Fluxo exemplar: O `Workspace_ID` fornecido no pedido determina de forma estrita o caminho base do sistema de ficheiros (ex: `/var/www/app/storage/{workspace_id}/`).

Validação: O input do caminho do ficheiro (`user_path_input`) é combinado com o caminho gerado pelo `Workspace_ID` validado.

Consistência Temporal e de Estado (Metadados):

Ao atualizar um documento, a data/hora da última modificação enviada pelo cliente é validada logicamente contra o estado atual na base de dados para prevenir sobreposições de ficheiros.

v2.1.3

O sistema documenta e aplica controlos rígidos sobre os limites operacionais da lógica de negócio. A ausência destes limites expõe a aplicação a ataques DoS através da exaustão de recursos. Os limites são estruturados em duas camadas complementares: restrições operacionais por utilizador e restrições globais da aplicação.

Tabela 6: Restrições operacionais por utilizador

Métrica	Limite Máximo	Ação em Caso de Violação	Justificação de Segurança
Tamanho de Ficheiro Markdown	2 MB por ficheiro	Rejeição do upload/gravação (<code>BusinessLimitError</code>)	Prevenir exaustão de memória no buffer de conversão para HTML.
Número de Workspaces	10 por utilizador	Bloqueio de criação no ecrã e API	Evitar a criação massiva de diretórias no sistema de ficheiros Linux.
Documentos por Workspace	100 documentos	Bloqueio de criação no ecrã e API	Limitar o crescimento descontrolado de inodes na diretoria do utilizador.

Tabela 7: Restrições globais da aplicação

Métrica / Recurso	Limite Máximo	Mecanismo de Controlo	Justificação de Segurança
Payload do Pedido HTTP	16 MB por request	<code>Flask.config</code> <code>['MAX_CONTENT_LENGTH']</code>	Impede o envio de ficheiros ou payloads gigantescos que esgotem a RAM do servidor.
Taxa Global de Pedidos (Rate Limit)	100 pedidos / minuto por IP	Middleware / Flask-Limiter	Proteção do servidor contra ferramentas de DoS.

6.2 Gestão de Sessões

v7.1.1

Em conformidade com o requisito v7.1.1 do ASVS 5.0, o ciclo de vida das sessões dos utilizadores é rigidamente controlado através de limites de tempo máximo absoluto. A arquitetura de autenticação recorre a JSON Web Tokens (JWT) encapsulados em cookies seguros, eliminando sessões locais no servidor (stateless), mas mantendo a capacidade de revogação.

Tabela 8: Tabela de Controlo de Sessão e Expiração de Tokens

Tipo de Controlo	Valor Definido	Mecanismo de Validação	Comportamento do Sistema
Tempo Máximo de Access Token	15 minutos	Verificação da timestamp de emissão do Access Token e renovação por janela deslizante (Sliding Window) do Refresh Token.	Se a aplicação não detetar atividade na API durante 30 minutos, o pedido de renovação automática é bloqueado e o utilizador é redirecionado para o ecrã de Login.
Tempo Máximo Absoluto (Absolute Maximum Lifetime)	12 horas	Reivindicação estrita de expiração gravada de forma imutável no payload do Refresh Token.	Independentemente da atividade contínua do utilizador, a sessão expira obrigatoriamente decorridas 24 horas desde a autenticação inicial, exigindo novas credenciais.

v7.1.2

A aplicação define e impõe um limite máximo para o número de sessões simultâneas permitidas para uma única conta de utilizador. Para operacionalizar este requisito numa arquitetura de autenticação assente em JSON Web Tokens (JWT). O comportamento adotado aquando do preenchimento da quota de sessões segue uma estratégia automatizada de revogação por ordem cronológica.

Tabela 9: Tabela de Controlo de Sessões Ativas Simultâneas

Parâmetro	Valor Definido	Estratégia de Resolução (Limite Atingido)	Justificação de Segurança
Máximo de Sessões Ativas	3 sessões em simultâneo	FIFO (First-In, First-Out) - Revogação automática da sessão mais antiga.	Permite que o utilizador mantenha o acesso no computador pessoal, de trabalho e num dispositivo móvel, bloqueando acessos em massa de terceiros.

7 Conclusão

O presente trabalho permitiu realizar uma avaliação abrangente da segurança do projeto, através da aplicação de diferentes técnicas de análise, incluindo SAST, DAST, Software Composition Analysis, análise de configuração de containers Docker e verificação de conformidade com o padrão ASVS.

Os resultados obtidos demonstram que, apesar de não terem sido identificadas vulnerabilidades críticas ao nível do código ou das dependências, existem pontos de melhoria relacionados com configurações de infraestrutura e boas práticas de segurança, especialmente ao nível de Docker e hardening dos containers.

De forma global, a aplicação apresenta um nível de segurança aceitável, com uma cadeia de dependências controlada e ausência de vulnerabilidades conhecidas nas bibliotecas analisadas.

Como trabalho futuro, recomenda-se a integração contínua das ferramentas utilizadas na pipeline de CI/CD, bem como a melhoria progressiva da conformidade com o padrão ASVS, de forma a aumentar a robustez e maturidade de segurança do sistema.